DOCUMENTATION

A Python graph language for the SNN tool

ar063 · 30 July 2026 · Draft

Abstract

Translating a circuit idea into *tools/snn* is cumbersome. A paper may describe a small motif in a paragraph, while its implementation requires manual work across CLI flags, model construction, tensor dimensions, training choices, recordings, and artifact handling. This proposal introduces *snnlang*: a typed Python library for constructing a spiking computation graph and, when needed, a narrow training specification. One compile operation validates them and writes a portable bundle containing *graph.json* and optional *training.json*. *tools/snn* executes that bundle through its existing CLI process boundary.

The proposal is deliberately smaller than a language for whole experiments, but it is not restricted to the present `COBANet` architecture. Experiment runners continue to own hypotheses, condition and seed grids, custom interventions, derived analysis, figures, and publication. An audit of the current code identifies three workloads that should test the design: trained MNIST and SHD classifiers, including deeper layers; coupled untrained E/I circuits with population-level recordings; and online confidence feedback that modulates g_L to trade decision speed for additional PING cycles. The implementation order is: draft the graph language, upgrade *tools/snn* to execute it, run one real experiment through both, and then prove that the combined system is flexible enough for the confidence experiment.

1. The existing separation of concerns

The repository intends to contain two different layers. *tools/snn*, documented in ar011, is the reusable simulation and training engine. Files under *experiments/* are scientific runners. The desired boundary is subprocesses and files, but the present code does not consistently maintain it.

```
experiment runner
      |
      | CLI invocation
      v
tools/snn/tool.py
      |
      | config, metrics, traces, weights
      v
experiment runner
      |
      | analysis, figures, numbers.json
      v
writings/expNNN.typ
```

This firewall is useful when it exists. It keeps reusable model science out of paper-specific code and prevents the simulator from accumulating every protocol ever used in the lab. The leaks are diagnostic: they identify missing reusable execution contracts more clearly than an abstract architecture exercise can.

1.1 What tools/snn owns

tools/snn owns operations that remain meaningful across experiments:

- neuron and synapse dynamics;
- graph execution and numerical integration;
- the standard training loop and surrogate-gradient implementation;
- checkpoint loading and saving;
- reusable input, recording, and perturbation primitives;
- generic rates, accuracy, and run metrics;
- deterministic seeding and run provenance;
- stable machine-readable outputs.

Its current use of *torch.compile* remains an internal execution detail. *snnlang* does not manage compilation caches or accelerator-specific artifacts.

1.2 What experiment runners own

A runner owns decisions whose meaning comes from one scientific question:

- the hypothesis and condition labels;
- parameter and seed grids;
- job ordering, caching, resumption, and dispatch;
- paper-specific data preparation;
- custom intervention streams built from earlier outputs;
- derived statistics and success criteria;
- figure design and *numbers.json*;
- staging and atomic publication.

Substantial runner code is not automatically a failure of abstraction. Code should move into a reusable tool only after a second use demonstrates a stable operation.

1.3 What the runners reveal

Existing runners make the boundary concrete. *exp022* defines a canonical family of trained MNIST cells. *exp041* computes spectra, gamma peaks, fits, and figures from tool outputs. *exp042* constructs experiment-specific inhibitory override streams from a reusable simulator primitive. *exp054* and *exp058* use untrained networks to study connectivity, synchrony, rhythmicity, and perturbation growth.

The newer SHD runners also expose failures in the current boundary. They import the CLI module in-process, replace the dataset directory, and construct networks directly for evaluation. *exp069* temporarily replaces `MetricsJsonl.write`, inspects its caller's local variables to obtain the live network, and saves a validation-selected checkpoint. This is ingenious in the same sense that opening a tin with a screwdriver is ingenious: it works, but it is evidence that the correct tool is missing.

The audit therefore identifies four engine seams needed before a general graph executor:

- an explicit data binding for dense samples and event streams, instead of fixed dataset paths;
- a standard checkpoint-selection policy or stable epoch event contract;
- evaluation of supplied event datasets without direct model construction;
- named recordings from every layer and population, rather than a privileged "primary" hidden layer.

These cases support one rule:

> The tool should understand one graph execution. The runner should understand why several executions constitute an experiment.

1.4 What the current model actually is

The present `COBANet` is a useful but narrow execution model:

- it builds a feedforward list of hidden E/I layers;
- E spikes alone feed the next hidden layer;
- each layer owns W_{EE}, W_{EI}, W_{IE}, and W_{II} recurrence;
- rate, mean-membrane, and cumulative-potential readouts are hard-coded modes;
- two hidden layers are already exercised by *exp071*;
- `train_leak` learns a static bounded leak per neuron, not an online feedback law;
- recording can capture all layers, although several consumers select only the deepest layer.

This means "support deeper networks" is not a distant grammar problem. The first graph schema should represent the existing sequential stack, but its ontology should be named populations and projections rather than `COBANet` constructor arguments. Coupling two independently named circuits is a genuinely new backend capability, even when neither circuit is trained. It is acceptable for the first compiler to emit graphs that the current backend reports as unsupported; the language and engine capability levels must be explicit rather than artificially identical.

1.5 Data is an execution binding, not graph structure

The standard trainer currently knows MNIST and SHD. MNIST is dense and Poisson encoded; SHD is an event stream binned lazily. The generic inference path still assumes 784 dense inputs, so SHD experiments work around it.

graph.json should declare typed input ports and output dimensions. *TrainSpec* should declare the target interface and encoder class. A runner should supply a small resolved data binding that maps actual train, validation, or evaluation sources onto those ports. Dataset splitting, sealing, and scientific cohort selection remain runner responsibilities. This avoids both extremes: hard-coding MNIST and SHD into the graph, or inventing a universal dataset language.

2. The proposed boundary

The proposal introduces one Python package with two related authoring objects:

```
snnlang.Network
snnlang.TrainSpec
```

Network describes forward computation. *TrainSpec* is an optional, narrow configuration for the standard trainer. It refers to parameters and outputs in a particular *Network*, but it is not part of that network and is not a second independent language.

One compile operation produces a bundle:

```
Python authoring
  Network
  optional TrainSpec
        |
        | snnlang.compile(...)
        v
ping.bundle/
  graph.json
  training.json
  manifest.json
        |
        | tools/snn CLI + data binding
        v
run artifacts
```

graph.json is the resolved executable graph. *training.json* records standard training policy and the digest of the exact graph against which it was validated. *manifest.json* binds the files into one convenient invocation unit.

A separate, resolved *data.json* is supplied when training or evaluating. It contains paths and representation metadata, not dataset contents. It belongs to an execution and may be replaced while the graph remains unchanged.

tools/snn depends on the versioned graph and training schemas, not on the *snnlang* Python package. Given an archived bundle, it must be possible to replay the run without installing the compiler that originally authored it.

2.1 Same repository, separate libraries

snnlang should begin in the same repository and Python environment as *tools/snn*, but as a sibling library:

```
tools/
    snn/
    snnlang/
schemas/
    snn-graph-v1.schema.json
    snn-training-v1.schema.json
```

Keeping both sides together makes early schema changes cheap. Keeping their package boundaries separate prevents the authoring library from becoming simulator internals. A separate repository or published project is justified only after another repository, simulator, or release cadence actually needs it.

3. The smallest useful authoring model

The first authoring surface is Python, not a custom textual grammar. Python already supplies composition, functions, loops, autocomplete, testing, and debugging.

```
import snnlang as snn
from snnlang import training

net = snn.Network("ping_classifier")

E = net.population(
    "E",
    size=1024,
    neuron=snn.COBA_LIF(...),
)
I = net.population(
    "I",
    size=256,
    neuron=snn.COBA_LIF(...),
)

ei = net.connect(
    E.spikes,
    I.excitatory,
    name="E_to_I",
    synapse=snn.AMPA(tau=...),
    weight=snn.Normal(0.5, 0.05),
    constraint=snn.NonNegative(),
)
ie = net.connect(
    I.spikes,
    E.inhibitory,
    name="I_to_E",
    synapse=snn.GABA(tau=9 * snn.ms),
```

```
    weight=snn.Normal(1.0, 0.1),
    constraint=snn.NonNegative(),
)

counts = net.reduce(
    E.spikes,
    operation="sum",
    over="time",
    name="E_spike_count",
)
logits = net.linear(
    counts,
    size=10,
    name="classifier",
)

net.output("class_logits", logits)
net.expose(E.spikes, I.spikes)
```

The graph contains populations, projections, ordinary transformations, parameters, outputs, and observables. It does not contain experiment conditions, sweeps, losses, optimizers, or figures.

3.1 Components and layers are authoring conveniences

A reusable component may shorten construction:

```
cell = snn.components.ping(
    net,
    name="cell",
    n_e=1024,
    n_i=256,
    tau_gaba=9 * snn.ms,
)
```

Components and layers expand into the same small graph vocabulary. The serialized graph may retain group metadata for reports, but it need not introduce a special executable `layer` primitive.

The same rule applies to readouts. A helper such as:

```
logits = snn.readouts.rate_classifier(
    net,
    source=cell.E.spikes,
    classes=10,
    name="class_logits",
)
```

expands into ordinary reduction, projection, and output operations. The first graph schema therefore has no special `head` ontology. Named outputs provide the interface needed by inference and training.

3.2 Parameters are not intrinsically selected for this run

The graph distinguishes a *Parameter* from a *Constant* and records structural constraints such as non-negativity, sparsity masks, or tying. It does not permanently declare that every parameter must be updated.

Selection belongs to a particular training specification:

```
train = snn.TrainSpec(
    objectives=[
        training.CrossEntropy(
            prediction=net.outputs["class_logits"],
            target="digit",
        ),
    ],
    parameter_groups=[
        training.ParameterGroup(
            [ei.weight, ie.weight],
            lr=1e-4,
        ),
        training.ParameterGroup(
            logits.parameters,
            lr=1e-3,
        ),
    ],
    regularizers=[
        training.UpperRatePenalty(
            signal=counts,
            threshold=1.0,
            strength=1e-4,
        ),
    ],
    optimizer=training.AdamW(),
    epochs=50,
)
```

The same graph can support readout-only training, recurrent fine-tuning, or inference without changing its structural identity.

3.3 Forward and backward concerns

Anything executed during inference belongs in the graph: populations, reductions, projections, classifier parameters, and named outputs. Anything used only to calculate or apply gradients belongs in *TrainSpec*: objectives, targets, parameter groups, regularizers, surrogate choice, clipping, and backward-only stop boundaries.

Checkpoints remain separate evolving state. They contain realised parameter values and, when resuming training, optimizer state. A checkpoint may initialise, resume, or partially map onto a graph, but its path is not part of the immutable graph.

3.4 Online feedback belongs to the combined execution system

Confidence-controlled leak is not a training recipe. At each timestep it computes evidence from the current readout, derives confidence, and modulates a population parameter before a later state update. The combined *snnlang* plus *tools/snn* system must support that causal loop, but version one need not give confidence or g_L modulation dedicated language constructs.

One eventual declarative spelling could be:

```
evidence = snn.readouts.cumulative(cell.E.spikes)
confidence = snn.signals.max_probability(evidence)

snn.controls.bounded_leak(
    source=confidence,
    target=cell.E,
    low_confidence_tau=30 * snn.ms,
    high_confidence_tau=10 * snn.ms,
    delay=1 * snn.step,
)
```

The spelling is provisional and is not a version-one requirement. Three implementation routes are legitimate:

- ordinary graph operations, if the initial vocabulary can express the loop cleanly;
- a generic stateful controller or modulatory-node extension implemented by *tools/snn*;
- custom experiment code using a small, stable runtime hook around a compiled graph.

Whichever route is chosen must state the source signal, transform, target, bounds, initial state, and whether the effect is immediate or delayed. A next-timestep default avoids an implicit algebraic loop. If controller parameters later become trainable, *TrainSpec* can select them like any other parameter.

The third route does not mean reaching into model locals or monkey-patching the training loop. It means an intentional engine extension point. The graph and checkpoint remain portable; the run manifest records the controller implementation and configuration. If the same controller pattern recurs, it can then be promoted into snnlang.

4. What remains outside snnlang

Neither *Network* nor *TrainSpec* should initially describe:

- experimental conditions or hypothesis labels;
- parameter and seed sweeps;
- matched-control searches;
- multi-stage curricula or alternating optimizers;
- arbitrary Python callbacks;
- experiment-specific adaptive interventions driven by intermediate results;

- paper-specific metrics;
- figures and publication;
- RunPod or Modal orchestration;
- automatic prose or claim generation.

A custom experiment may use *graph.json* with custom Python training code and omit *training.json*. A feature enters *TrainSpec* only when the standard *tools/snn* trainer supports it and repeated experiments share it.

A paper-specific rule that decides which condition to run next remains in the runner. A within-trial control loop must execute inside the simulator runtime, whether authored directly in the graph or attached through a stable extension point.

5. Compilation and static analysis

Compilation is pure and deterministic:

```
bundle = snn.compile(
    net,
    training=train,
    target="tools/snn",
)
bundle.write("ping.bundle")
```

It performs no simulation, accelerator allocation, realised random initialization, or mutation of *tools/snn* globals. Python object references become stable graph identifiers at serialization.

5.1 Hard validation

The early compiler can provide substantial value before execution:

- schema and version validation;
- unique names and resolved references;
- tensor and projection shapes;
- physical units;
- port compatibility;
- graph reachability from inputs to outputs;
- disconnected populations and unused projections;
- Dale and sign-constraint compatibility;
- parameter inventory and optimizer-group membership;
- objective and target compatibility;
- differentiable reachability from each objective;
- surrogate-gradient coverage on objective paths;
- feedback-cycle delays and a deterministic within-step update order;
- controller output units, target compatibility, and declared bounds;
- checkpoint names and shapes;
- backend capability checks;
- deterministic canonical serialization.

Example diagnostics should identify the object and failed relation:

```
error E203: projection "E_to_I" has shape [800, 200]
  expected [1024, 256] from E.spikes -> I.excitatory

error E501: selected parameter "I_to_E.weight"
  has no differentiable path to objective "classification"
```

5.2 Reports and estimates

The analyser can also report:

- population and projection tables;
- strongly connected components and recurrent motifs;
- feedforward, recurrent, and modulatory paths between named populations;
- parameter counts and selected versus frozen parameters;
- expected sparse edge counts and fan-in;
- approximate parameter, optimizer, state, recording, and BPTT memory;
- semantic differences between two bundles;
- whether two runs share a graph but differ in training policy;
- whether a bundle contains unresolved environment-dependent defaults.

Numerical risk checks should be warnings rather than proofs:

```
warning W311: tau_gaba=0.2 ms is only two timesteps
  at dt=0.1 ms; the decay may be poorly resolved
```

5.3 Visual graph reports

snnlang should optionally render a compiled bundle as a polished circuit diagram. This is a compiler report, not a simulator feature:

```
bundle.visualise(
    "network.svg",
    view="circuit",
)
bundle.visualise(
    "network.png",
    view="circuit",
    scale=2,
)
```

The implementation should generate styled DOT from *graph.json*, use Graphviz for layout, and treat SVG as the canonical visual output. PNG and PDF are derived publication formats. Graphviz is responsible for ranks, routing, and crossing reduction; *snnlang* remains responsible for visual meaning.

The visual grammar should be stable:

- rounded cards for populations, with size and neuron model;
- restrained, consistent colours for excitatory, inhibitory, input, output, and modulatory nodes;
- solid excitatory projections, inhibitory bar endings, and dashed modulatory paths;

- softly bounded clusters for layers, PING cells, and user-defined components;
- feedback routed separately from the primary feedforward direction;
- optional edge width or annotation for density, fan-in, delay, or weight summary;
- subtle marks for trainable parameters and stop-gradient boundaries.

One overloaded picture is not the goal. The renderer should offer at least:

- `circuit`, a paper-ready population and projection view;
- `training`, showing outputs, objectives, trainable groups, and gradient boundaries;
- `expanded`, showing lower-level operations and stateful nodes for debugging.

Components are collapsed by default and expandable on request. Parallel projections may be grouped, default parameters suppressed, and a selected path highlighted. If a graph is too dense for a legible static image, the renderer should warn and require filtering or collapsing rather than emit decorative spaghetti. Layout, colours, fonts, node ordering, and identifiers must be deterministic so an unchanged bundle produces an unchanged diagram.

The canonical SVG should retain stable object identifiers and classes. This permits tooltips or interactive inspection later without changing the graph schema. Visualisation failure must never make an otherwise valid bundle unexecutable.

5.4 What static analysis does not promise

The early analyser does not predict whether gamma emerges, its frequency, a Hopf threshold, firing rates, accuracy, or successful optimization. Those are dynamical claims. Later restricted analysers may attach approximate semantics to supported motifs, but every result must state its assumptions and unsupported components.

6. Runtime and CLI boundary

Experiment runners should receive a typed helper while retaining process isolation:

```
from experiments.helpers.snn import run_training

run_training(
    bundle=bundle_path,
    seed=seed,
    out_dir=cell_dir,
)
```

The helper invokes:

```
uv run python tools/snn/tool.py train \
  --bundle ping.bundle \
  --data data.json \
  --seed 42 \
  --out-dir cell/
```

Inference needs only the graph and optional checkpoint:

```
uv run python tools/snn/tool.py sim \
  --graph ping.bundle/graph.json \
  --weights weights.pth \
  --out-dir inference/
```

Scientific graph and standard training settings live in validated files rather than long argument lists. The data binding carries resolved sources. CLI arguments carry paths, seeds, output locations, and lifecycle controls.

The process boundary remains valuable because *tools/snn* currently uses mutable module-level configuration for sizes, timestep, and other execution state. A fresh process contains failures, accelerator memory, compiler state, and globals. A small request-based Python API may eventually sit underneath the CLI, but ordinary runners need not import the engine directly.

Existing flag-driven experiments remain supported. The manifest path is additive:

```
legacy flags -> existing builder -> execution
graph bundle -> graph loader     -> execution
```

This permits gradual adoption without rewriting the historical experiment corpus.

7. Staged implementation

The implementation should follow four project steps. Existing experiments provide evidence and regression cases, but they do not define the ceiling of the language.

Step 1: implement the first snnlang draft

Implement a modest, genuinely graph-shaped Python package:

```
tools/snnlang/
    network.py
    parameters.py
    operations.py
    training.py
    data.py
    validation.py
    compile.py
    backends/tools_snn.py
```

Its core vocabulary should include typed inputs, named populations, projections, neuron and synapse specifications, delays, parameters and constants, named signals and observables, and ordinary readout operations. *TrainSpec* adds objectives, parameter groups, regularizers, and standard optimization policy. A narrow data interface distinguishes dense samples from event streams.

The draft may describe more topology than *tools/snn* can currently execute. Compilation should separate language validity from backend support:

```
graph.valid             == true
backend.tools_snn.valid == false
missing_capabilities    == ["arbitrary_feedback"]
```

That separation prevents the current simulator class hierarchy from becoming the language specification.

Acceptance gate:

1. One-layer, deeper feedforward, recurrent, and feedback topologies are representable.
2. The compiler produces deterministic, accelerator-independent graph, training, analysis, and visual reports without importing *tools/snn*.
3. A representative multilayer or coupled graph produces a legible `circuit.svg` and high-resolution PNG with collapsed components and distinguishable feedback.
4. Invalid shapes, references, units, parameter groups, and objectives fail before execution.
5. Backend capability failures are precise and separate from graph errors.

Step 2: upgrade tools/snn to execute the draft
Add graph, training, and data-binding loaders to *tools/snn*. The first implementation may lower legacy-shaped graphs into `COBANet`, but arbitrary named population graphs require a new graph executor rather than ever more constructor flags. Preserve the numerical kernels that are already useful; replace the rigid orchestration around them.

Repair the execution seams discovered by the audit at the same time: supplied datasets, event-stream evaluation, checkpoint-selection policy, named recordings, and stable runtime extension points.

Acceptance gate:

1. Legacy CLI behaviour remains unchanged.
2. A trained MNIST cell, a trained SHD cell, the existing two-layer SHD model, and an untrained E/I simulation preserve behaviour within declared tolerances.
3. A runner can select "maximum validation accuracy, then minimum validation loss, then earliest epoch" declaratively.
4. An SHD split can be supplied without copying it into a fixed temporary directory.
5. Named populations can be connected through feedforward and feedback projections and recorded independently.

Editing *tools/snn* requires explicit project permission. The authoring library and schemas can be developed before that integration point.

Step 3: create the first native experiment
The first experiment should use snnlang because the graph representation materially helps, not merely to prove that old flags can be written as JSON. A strong candidate is two untrained PING or balanced E/I circuits connected by a controllable projection, because it exercises named populations, arbitrary coupling, delays, feedback, independent drives, and population recordings without simultaneously debugging autograd.

Acceptance gate:

1. Two PING or balanced E/I components can be coupled without custom simulator code.

2. Removing, reversing, or delaying an inter-circuit projection is a graph edit.
3. Synthetic spike tests verify delays and feedback update order exactly.
4. The runner can compare within-population and between-population synchrony from named outputs.
5. The compiled bundle is retained with the experiment artifacts.

Synchrony, coherence, phase locking, and cross-correlation remain runner analyses over emitted spikes and traces. Making the first experiment untrained isolates graph execution from autograd. If a trained experiment offers more immediate scientific value when this stage begins, it can be substituted without changing the architectural gate.

Step 4: support the confidence experiment
Make the combined system support an MNIST classifier whose online confidence controls g_L, slowing low-confidence dynamics to permit more PING cycles. First attempt composition from ordinary graph operations. If that makes the language contorted, implement a generic controller node or stable runtime hook in *tools/snn*. Do not add a `ConfidenceToLeak` language primitive merely because one experiment wants it.

Acceptance gate:

1. The controller sees only evidence available up to the declared timestep.
2. Fixed-confidence tests produce the expected bounded g_L or τ_m trajectory.
3. A one-step feedback delay has an exact causal test.
4. Runs report decision time, PING cycles before decision, confidence, accuracy, and spike-count or energetic cost.
5. A fixed-leak control uses the same graph with the controller disabled.

Additional cycles may improve confidence, do nothing, or amplify a wrong attractor. The language makes the experiment easy to state; it cannot guarantee the desired result.

Later: trainable coupled graphs and gamma components
Once coupled forward graphs are stable, allow *TrainSpec* to select parameters across them and place explicit stop-gradient boundaries. This is where scoped backpropagation becomes useful rather than decorative.

PING, ING, and balanced asynchronous helpers may form a gamma component library, but they expand into explicit physical parameters. No component may assert that gamma exists solely from its label; the run must measure rhythmicity.

Later: extension interface
Repeated demand may justify user-defined composites and backend primitives. An extension declares ports, state, parameters, constraints, differentiability, serialization, and backend support. The first implementation should not design this interface speculatively.

Later: optional reduced semantics
A separate analysis layer may attach approximate transfer or mean-field descriptions to restricted components. Hopf and DMFT-level prediction are not acceptance criteria for the executable language.

Horizon: general spiking-network grammar
Backend independence, simulator portability, symbolic rewrites, graphical editing, and bifurcation backends remain the horizon. They are promoted only after several *tools/snn* experiments demonstrate that the smaller graph language is genuinely useful.

8. Migration policy

Completed experiments are scientific records, not merely old application code. They should remain executable through their existing runners.

- New experiments use *snnlang* once the needed feature set exists.
- A small MNIST, SHD, and untrained simulation suite becomes the conformance set.
- An old experiment migrates when it is materially extended.
- A migrated version is treated as a new implementation and compared explicitly.
- Custom training or intervention code remains acceptable when it is genuinely experiment-specific.

Mass retrofitting would risk changing defaults, seed handling, initialization, or simulator configuration while producing reassuringly similar figures. Architectural purity is not worth damaged provenance.

9. Success criterion

The first implementation milestone is:

> *snnlang* independently compiles a deterministic, statically checked population graph; the upgraded *tools/snn* executes that bundle through the CLI; and one native experiment uses the pair without importing or monkey-patching engine internals.

The confidence-to-g_L experiment is the next system-level acceptance test. It succeeds whether the feedback is expressed through ordinary graph operations, a generic controller extension, or a stable runtime hook. The important constraint is that it composes with a compiled graph, remains reproducible, and does not require surgery on simulator internals.

Appendix A. Bundle and artifact lifecycle

A.1 Construction and compilation

An experiment-specific definition may begin inside its runner:

```
def make_bundle(tau_gaba_ms: float):
    net = make_ping_classifier(
        tau_gaba_ms=tau_gaba_ms,
    )
    train = make_standard_training(net)
    return snnlang.compile(
        net,
        training=train,
        target="tools/snn",
    )
```

The first use remains local. A repeated component may later move into a reusable circuit module. Reuse must be observed rather than predicted.

Compilation writes:

```
ping-tg9.bundle/
    manifest.json
    graph.json
    training.json
    reports/
        circuit.svg
        circuit.png
```

The executable files contain data only. They contain no Python callables, live instances, pickled authoring objects, accelerator tensors, or *torch.compile* products. The optional *reports/* directory contains derived human-readable views and may be regenerated exactly from the bundle.

A.2 Bundle contract

graph.json contains:

- populations, operations, ports, and projections;
- dynamics and structural parameters;
- initializer specifications;
- constants and optimizable parameters;
- constraints, masks, and ties;
- named outputs and exposed observables;
- grouping metadata for source-level components.

training.json contains:

- the exact graph digest;
- the expected data interface and encoder supported by the standard trainer;
- objectives, targets, and regularizers;
- optimizer parameter groups;
- surrogate-gradient and clipping settings;
- epochs and an explicit standard checkpoint-selection policy.

A resolved *data.json* contains:

- the input representation, such as dense samples or timestamped events;
- train, validation, or evaluation source paths;
- the mapping from source fields to graph inputs and training targets;
- shape, class-vocabulary, split, and content digests needed for validation and provenance.

It does not decide how a scientific split was created. That remains runner code.

manifest.json binds the files:

```
{
  "schema": "snnlang.bundle/v1",
  "graph": {
    "path": "graph.json",
    "sha256": "..."
  },
  "training": {
    "path": "training.json",
    "sha256": "...",
    "graph_sha256": "..."
  }
}
```

A simulation-only bundle may omit *training.json*. *tools/snn* rejects missing files, unsupported schema versions, digest mismatches, or backend capabilities absent from the graph.

A.3 Run state

config.json, written by *tools/snn*, records the resolved execution:

- execution mode, seed, device, timestep, and duration;
- graph and training schema versions and digests;
- data-binding, input, and checkpoint paths and digests;
- requested outputs;
- code and environment provenance.

A checkpoint contains evolving parameter values and optional optimizer state. Initializing, resuming, fine-tuning, and inference are invocation choices rather than changes to *graph.json*.

A.4 Scratch layout

Each experiment uses regenerable scratch separately from its committed publication record:

```
temp/experiments/expNNN/
    bundles/
        ping-tg4p5.bundle/
        ping-tg9.bundle/
        ping-tg18.bundle/
    reports/
        ping-tg4p5.svg
        ping-tg9.svg
        ping-tg18.svg
    cells/
        ping-tg4p5-seed42/
        ping-tg4p5-seed43/
        ping-tg9-seed42/

artifacts/data/expNNN/
```

A runner compiles once per unique graph and training policy, not once per seed. It may reuse that bundle with different validated data bindings.

A.5 Execution

The runner uses a typed subprocess helper:

```
run_training(
    bundle=bundles[condition],
    data=data_binding,
    seed=seed,
    out_dir=cell_dir(condition, seed),
)
```

At the shell boundary:

```
tools/snn train \
  --bundle ping-tg9.bundle \
  --data data.json \
  --seed 42 \
  --out-dir cells/ping-tg9-seed42
```

The tool copies the exact files it executed into the run directory:

```
cells/ping-tg9-seed42/
    graph.json
    training.json
    data.json
    config.json
    metrics.json
    weights.pth
    output.log
    run.jsonl
    run.sh
```

Copying prevents later edits to a source bundle from changing the apparent meaning of existing weights and metrics.

A.6 Publication

The committed experiment record deduplicates descriptions while retaining the mapping from every reported cell:

```
artifacts/data/expNNN/
    numbers.json
    bundle_manifest.json
    graphs/
        ping-tg4p5.json
        ping-tg9.json
        ping-tg18.json
    diagrams/
        ping-tg4p5.svg
        ping-tg9.svg
        ping-tg18.svg
    training/
        train-tg4p5.json
        train-tg9.json
        train-tg18.json
    figure.svg
    raster.png
```

The manifest maps each cell to graph and training digests:

```
{
  "cells": [
    {
      "condition": "tg9",
      "seed": 42,
      "graph": "ping-tg9",
      "training": "train-tg9"
    }
  ]
}
```

Compiler caches and accelerator-specific products are disposable implementation state and are not published scientific artifacts.

A.7 Atomic runner lifecycle

```
def main():
    run_id = next_run_id(SLUG)

    with published_run(
        SLUG,
        run_id,
        scale=SCALE,
    ) as (scratch, staging):

        bundles = {}

        for condition in CONDITIONS:
            bundle = make_bundle(condition)
```

```
            path = (
                scratch
                / "bundles"
                / f"{condition}.bundle"
            )
            bundle.write(path)
            bundles[condition] = path

        for condition in CONDITIONS:
            for seed in SEEDS:
                run_training(
                    bundle=bundles[condition],
                    data=data_binding(condition),
                    seed=seed,
                    out_dir=cell_dir(
                        condition,
                        seed,
                    ),
                )

        rows = analyse_cells(
            CONDITIONS,
            SEEDS,
        )
        render_figures(rows, staging)
        publish_descriptions(
            bundles,
            staging,
        )
        write_bundle_manifest(
            bundles,
            CONDITIONS,
            SEEDS,
            staging,
        )
        write_numbers(
            staging,
            run_id=run_id,
            duration_s=duration,
            payload=summarise(rows),
        )
```

The final artifact directory is replaced only after compilation, execution, analysis, plotting, and summary generation all succeed. A failed run cannot publish new graph descriptions beside figures from an older run.